

Universidad Autónoma de Santo Domingo

Facultad de Ingeniería y Arquitectura

Asignatura: INF-8237 Ciencia de Datos I

Caso práctico 2

**CRUD/ORM nativo basado en POO y estructuras de datos
en MySQL Sakila**

Equipo: UASDVirtual

Integrante: Marlenis Judith Concepción Cuevas

Facilitador: Dr. Silverio del Orbe

Fecha: 4 de junio de 2026

Tabla de contenido

Resumen	3
Abstract	4
Introducción	5
Marco de referencia	6
Descripción del caso práctico	7
Arquitectura del sistema	8
Implementación del CRUD/ORM	9
Pruebas y resultados	10
Conclusiones	11
Referencias	12
Anexos	13

Resumen

Este informe presenta el desarrollo del Caso práctico 2 de la Unidad 2 de Ciencia de Datos I, consistente en la creación de un CRUD/ORM nativo en Python para operar sobre la base de datos Sakila de MySQL. La solución utiliza programación orientada a objetos para representar entidades relacionales como países, ciudades, películas e inventario mediante clases de dominio, repositorios, controladores, servicios y un contexto de base de datos. El proyecto incorpora estructuras de datos como listas de entidades, diccionarios para caché y un historial de operaciones para evidenciar el uso de abstracciones propias de Python. Para facilitar la ejecución por los integrantes del equipo, se prepararon scripts de inicio para Mac, Linux y Windows que levantan MySQL en Docker, importan Sakila, verifican la conexión y abren el menú interactivo del CRUD. Además del flujo crear, leer, actualizar y eliminar, el trabajo incluye importación y exportación en CSV y JSON, diez consultas SQL, restricciones de integridad mediante unique constraints y métricas descriptivas fundamentales como media, rango, desviación estándar, varianza y covarianza. Los resultados demuestran una arquitectura modular y escalable, cercana al patrón ORM, donde los cambios del modelo pueden mantenerse separados de la conexión, la lógica de aplicación y la interfaz de consola. La propuesta fortalece la comprensión práctica de bases de datos relacionales, consumo programático de datos y organización de código reutilizable para proyectos de ciencia de datos.

Palabras clave

CRUD, ORM, Sakila, MySQL, Python, estructuras de datos.

Abstract

This report presents the development of Practical Case 2 for Data Science I, focused on building a native CRUD/ORM in Python over the MySQL Sakila sample database. The solution applies object-oriented programming to represent countries, cities, films and inventory as domain entities, while repositories, controllers, services and a database context organize the application flow. The project also uses Python data structures such as entity lists, dictionaries for cache management and an operation history. To support reproducible execution, the repository includes quickstart scripts for Mac, Linux and Windows that start MySQL with Docker, import Sakila, test the connection and launch the interactive CRUD menu. The work also includes CSV and JSON import/export, ten SQL queries, integrity constraints and descriptive metrics such as mean, range, standard deviation, variance and covariance. Overall, the solution demonstrates a modular structure that separates persistence, business flow and user interaction.

Keywords

CRUD, ORM, Sakila, MySQL, Python, data structures.

Introducción

La manipulación de datos en ciencia de datos requiere comprender tanto la estructura de los datos como los mecanismos usados para consultarlos, transformarlos y validarlos. En este caso práctico se utiliza Sakila, una base de datos de ejemplo de MySQL, para construir una solución que conecta Python con un sistema gestor relacional y permite ejecutar operaciones CRUD desde una interfaz de consola.

La actividad responde al objetivo de crear un CRUD/ORM nativo basado en programación orientada a objetos y estructuras de datos. El enfoque ORM permite representar tablas como objetos de dominio y encapsular las operaciones SQL en repositorios y servicios, reduciendo la dependencia directa entre la interfaz de usuario y las consultas de base de datos (Fowler, 2002).

Marco de referencia

Un CRUD agrupa las operaciones crear, leer, actualizar y eliminar, necesarias para administrar registros persistentes. En bases de datos relacionales, estas acciones se implementan mediante instrucciones SQL como INSERT, SELECT, UPDATE y DELETE. Oracle (s.f.) presenta Sakila como una base de datos de ejemplo útil para practicar relaciones entre tablas de un sistema de alquiler de películas.

Python facilita este tipo de solución por su soporte para clases, dataclasses, listas, diccionarios y módulos reutilizables. La documentación oficial de Python describe las clases como un mecanismo para combinar datos y comportamiento dentro de una misma estructura (Python Software Foundation, s.f.). Docker, por su parte, permite ejecutar servicios como MySQL en contenedores reproducibles, lo que reduce diferencias entre equipos de trabajo (Docker, s.f.).

Descripción del caso práctico

El proyecto gestiona las entidades country, city, film e inventory de Sakila. Estas entidades cubren los requisitos de países, ciudades, películas e inventario planteados en la asignación. La conexión se configura mediante variables de entorno y puede ejecutarse con MySQL local o con el contenedor Docker preparado para el proyecto.

El alcance técnico incluye diez consultas SQL, operaciones CRUD, importación y exportación de datos en CSV y JSON, métricas descriptivas y restricciones de integridad para mejorar la calidad de los datos. La solución se orienta a demostrar dominio de estructuras de datos, separación de responsabilidades y consumo programático de una base de datos relacional.

Arquitectura del sistema

- `db.py`: administra la conexión a MySQL y el manejo transaccional.
- `dbcontext.py`: centraliza repositorios, caché e historial de operaciones.
- `models.py`: define entidades y `ModelCollection` como lista de objetos.
- `repositories.py`: implementa CRUD genérico y repositorios concretos.
- `controllers.py` y `services.py`: separan el flujo de aplicación de la persistencia.
- `structures.py`: contiene caché de entidades e historial de consultas.
- `metrics.py` y `reports.py`: calculan media, rango, varianza, desviación y covarianza.
- `import_export.py`: permite importar y exportar datos CSV y JSON.
- `main.py`: ofrece el menú de consola para ejecutar la demostración.

Implementación del CRUD/ORM

La capa BaseRepository recibe una clase de modelo y construye consultas parametrizadas para crear, buscar por identificador, listar, actualizar y eliminar registros. Cada repositorio concreto hereda esa funcionalidad para una entidad específica. La interfaz de consola permite seleccionar países, ciudades, películas e inventario, introducir datos y observar el resultado devuelto como objeto.

La implementación evita credenciales escritas directamente en el código y usa variables de entorno. Además, los scripts quickstart de Mac, Linux y Windows simplifican la preparación de MySQL/Sakila con Docker para que el equipo pueda repetir la demostración en diferentes computadoras.

Pruebas y resultados

Las pruebas unitarias disponibles validan estructuras de datos usadas por el proyecto. En la verificación local se ejecutó pytest sobre el Caso 2 y el resultado fue de dos pruebas aprobadas. También se comprobó la conexión a MySQL Sakila por Docker, obteniendo conexión exitosa, nombre de base de datos sakila y versión MySQL 8.0.46.

- Create: creación de países de prueba desde el menú.
- Read: búsqueda por ID y listado de entidades.
- Update: modificación de registros seleccionados.
- Delete: eliminación de registros de prueba.
- Métricas: cálculo descriptivo sobre longitud, tarifa y costo de películas.
- SQL: ejecución de diez consultas y script de unique constraints.

Conclusiones

La solución cumple el objetivo principal de implementar un CRUD/ORM nativo sobre Sakila usando POO y estructuras de datos. La separación entre contexto, modelos, repositorios, controladores y servicios facilita ampliar el sistema a nuevas tablas sin reescribir toda la aplicación. La automatización con Docker mejora la reproducibilidad para los integrantes del equipo y permite generar evidencias reales de ejecución.

Como mejora futura, se recomienda ampliar las pruebas de integración, agregar validaciones de negocio más detalladas antes de insertar o actualizar datos y completar anexos con capturas finales de cada operación requerida por la rúbrica.

Referencias

Docker. (s.f.). Docker documentation. <https://docs.docker.com/>

Fowler, M. (2002). Patterns of enterprise application architecture. Addison-Wesley.

Oracle. (s.f.). Sakila sample database. MySQL documentation.
<https://dev.mysql.com/doc/sakila/en/>

Python Software Foundation. (s.f.). The Python tutorial.
<https://docs.python.org/3/tutorial/>

Anexos

- Anexo A. Código fuente relevante: carpeta src/ del Caso práctico 2.
- Anexo B. Evidencias de ejecución: capturas de conexión, CRUD, métricas y pruebas.
- Anexo C. Scripts SQL: consultas y unique constraints incluidos en la carpeta sql/.
- Anexo D. Uso de agentes IA: apoyo en documentación, automatización y revisión.